# Práctica 11

¿calcular cuántos años más podremos seguir utilizando esta forma de contar?

¿Qué significa que el tiempo medido sea 0? ¿A partir de qué tamaño de problema (n) se obtienen tiempos válidos >=50 msg?

Que la medida ha sido tan pequeña que no se ha podido medir con precisión, entrando en acción el recogedor de basura en java. En mi caso, con mi ordenador de procesador i5 12400, el programa empieza a dar tiempos válidos a partir del tamaño de problema n = 13000000.

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

Pues que el tiempo, aproximadamente, también se ve multiplicado por 2.

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.)

Como en el caso anterior, esta es una influencia directa en el resultado de los tiempos del problema.

¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n) de la operación suma?

Cumple perfectamente, pues cada tiempo es aproximadamente, una vez medidos con claridad, el doble que el anterior, cumpliendo con la linealidad.

A partir de lo visto en Vector4.java midiendo los tiempos de suma, realice las tres siguientes Clases: Vector5.java para medir los tiempos del máximo, Vector6.java para medir los tiempos de coincidencias1 y Vector7.java para medir los tiempos de coincidencias2. Con los tiempos obtenidos (en milisegundos) de las clases anteriores, rellene las dos tablas:

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

| n | t suma | t máximo |
|---|---|---|
| 10000 | 0,0413 | 0,06 |
| 20000 | 0,081 | 0,118 |
| 40000 | 0,156 | 0,235 |
| 80000 | 0,311 | 0,464 |
| 160000 | 0,63 | 0,93 |
| 320000 | 1,24 | 1,88 |
| 640000 | 2,49 | 3,73 |
| 1280000 | 5,2 | 7,5 |

| | | |
|---|---|---|
| 2560000 | 10 | 15,2 |
| 5120000 | 20,9 | 30,7 |
| 10240000 | 40,6 | 61 |
| 20480000 | 81 | 120 |
| 40960000 | 162 | 257 |
| 81920000 | 322 | 490 |

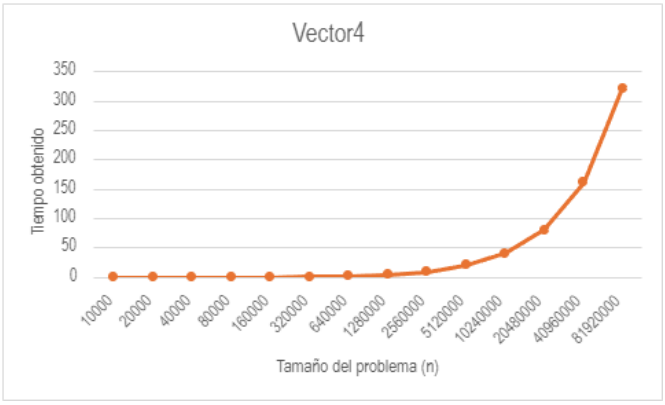
| n | t coincidencias1 | t coincidencias2 |
|---|---|---|
| 10000 | 552 | 0,059 |
| 20000 | 2084 | 0,122 |
| 40000 | 8333 | 0,248 |
| 80000 | 33332 | 0,481 |
| 160000 | FdT | 0,96 |
| 320000 | FdT | 2,1 |
| 640000 | FdT | 3,88 |
| 1280000 | FdT | 7,9 |
| 2560000 | FdT | 15,8 |
| 5120000 | FdT | 31,7 |
| 10240000 | FdT | 62 |
| 20480000 | FdT | 128 |
| 40960000 | FdT | 247 |
| 81920000 | FdT | 498 |

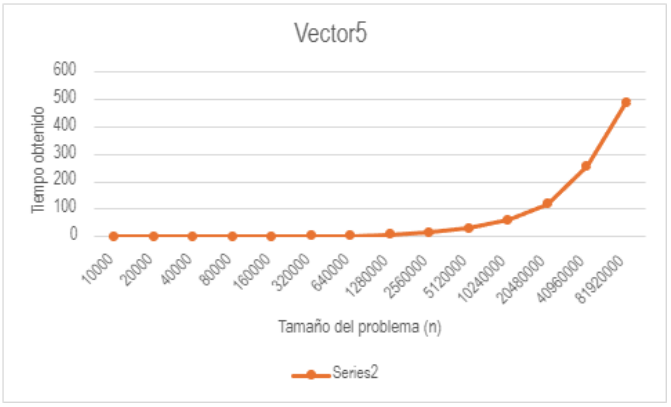
Una vez rellenadas ambas tablas, concluya si los tiempos obtenidos cumplen con lo esperado, dada la complejidad temporal computacional de las diferentes operaciones

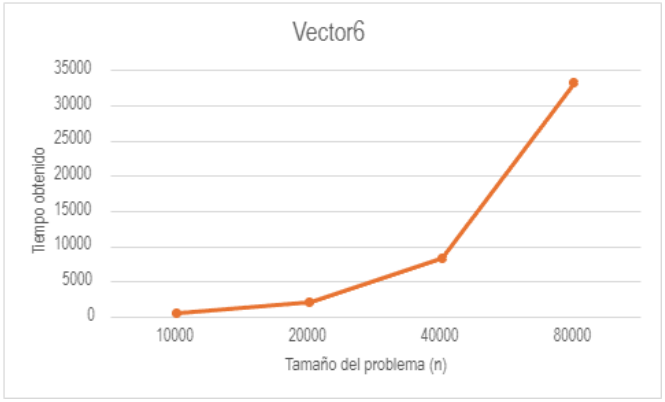
En estos tiempos, podemos apreciar que en la mayoría, específicamente la suma, el máximo y coincidencias2, cada tiempo vale, aproximadamente, el doble que el anterior, lo cual va acorde al hecho de que los tres cuentan con una complejidad O(n).

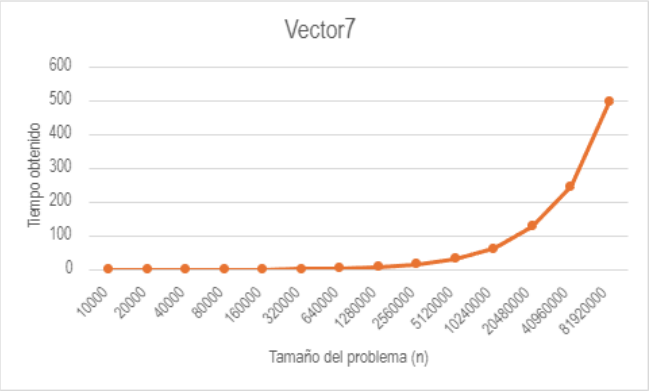
Sin embargo, coincidencias1 destaca por tener un tiempo claramente peor que el de su otra versión, a pesar de que, como se puede comprobar ejecutando el código dado, cumplen la misma función. La explicación de esto está en el código de la función, pues esta versión usa dos bucles anidados poco óptimos, a diferencia de su versión homónima, dándole por ende una complejidad O(n^2) y explicando el aumento drástico de tiempos, y el incremento de aproximadamente multiplicación por 4 entre cada uno.

# Sección de gráficas de los algoritmos empleados




Vector7
600
500
400
300
200
100
0
Tiempo obtenido
10000
20000
40000
80000
160000
320000
640000
1280000
2560000
5120000
10240000
20480000
40960000
81920000
Tamaño del problema (n)